

Exercise 9 – NgRx State Management (Store, Actions, Reducers, Selectors)

Install NgRx

```
npm install @ngrx/store @ngrx/store-devtools
```

Generate Store Files

```
ng generate interface store/course-state
```

```
ng generate action store/course
```

```
ng generate reducer store/course
```

```
ng generate selector store/course
```

course.model.ts

```
export interface Course{  
  id:number;  
  name:string;  
  code:string;  
  credits:number;  
  faculty:string;  
}
```

course.actions.ts

```
import { createAction,props } from '@ngrx/store';  
import { Course } from '../models/course.model';  
  
export const loadCourses=createAction(  
  '[Course] Load Courses'  
);
```

```
export const loadCoursesSuccess=createAction(
  '[Course] Load Courses Success',
  props<{courses:Course[]}>()
);

export const addCourse=createAction(
  '[Course] Add Course',
  props<{course:Course}>()
);

export const removeCourse=createAction(
  '[Course] Remove Course',
  props<{id:number}>()
);
```

course.reducer.ts

```
import { createReducer,on } from '@ngrx/store';
import * as CourseActions from './course.actions';
import { Course } from './models/course.model';
export interface CourseState{
  courses:Course[];
}
export const initialState:CourseState={
  courses:[]
};
export const courseReducer=createReducer(
  initialState,
```

```

on(CourseActions.loadCoursesSuccess,(state,{courses})=>({
  ...state,
  courses
})),
on(CourseActions.addCourse,(state,{course})=>({
  ...state,
  courses:[...state.courses,course]
})),
on(CourseActions.removeCourse,(state,{id})=>({
  ...state,
  courses:state.courses.filter(c=>c.id!==id)
}))
);

```

course.selectors.ts

```

import { createFeatureSelector,createSelector } from '@ngrx/store';
import { CourseState } from './course.reducer';

export const selectCourseState=
createFeatureSelector<CourseState>('courses');

export const selectAllCourses=
createSelector(
  selectCourseState,
  state=>state.courses
);

export const selectCourseCount=

```

```
createSelector(  
  selectAllCourses,  
  courses=>courses.length  
);
```

app.config.ts

```
import { ApplicationConfig } from '@angular/core';  
import { provideRouter } from '@angular/router';  
import { provideStore } from '@ngrx/store';  
import { provideStoreDevtools } from '@ngrx/store-devtools';  
import { routes } from './app.routes';  
import { courseReducer } from './store/course.reducer';  
  
export const appConfig:ApplicationConfig={  
  providers:[  
    provideRouter(routes),  
    provideStore({  
      courses:courseReducer  
    }),  
    provideStoreDevtools()  
  ]  
};
```

course-list.component.ts

```
import { Component } from '@angular/core';  
import { Store } from '@ngrx/store';  
import { Observable } from 'rxjs';  
import { Course } from '../models/course.model';
```

```
import * as CourseActions from '../store/course.actions';
import * as CourseSelectors from '../store/course.selectors';

@Component({
  selector: 'app-course-list',
  standalone: true,
  templateUrl: './course-list.component.html',
  styleUrls: ['./course-list.component.css']
})
export class CourseListComponent {
  courses$: Observable<Course[]>;
  courseCount$: Observable<number>;
  constructor(private store: Store) {
    this.courses$ =
      this.store.select(
        CourseSelectors.selectAllCourses
      );
    this.courseCount$ =
      this.store.select(
        CourseSelectors.selectCourseCount
      );
  }

  loadCourses() {
    const data: Course[] = [
      {
        id: 1,
        name: 'Angular',

```

```
code:'ANG101',
credits:4,
faculty:'Mr. Kumar'
},
{
id:2,
name:'React',
code:'RCT201',
credits:3,
faculty:'Mrs. Priya'
},
{
id:3,
name:'Spring Boot',
code:'SPR301',
credits:5,
faculty:'Mr. Raj'
}
];
this.store.dispatch(
CourseActions.loadCoursesSuccess({
courses:data
})
);
}
addCourse(){
this.store.dispatch(
CourseActions.addCourse({
```

```
course:{
  id:4,
  name:'Java',
  code:'JAVA101',
  credits:4,
  faculty:'Mr. Arun'
}
})
);
}

removeCourse(){
  this.store.dispatch(
    CourseActions.removeCourse({
      id:2
    })
  );
}
}
```

course-list.component.html

```
<h2>NgRx Course Store</h2>

<button (click)="loadCourses()">Load Courses</button>

<button (click)="addCourse()">Add Course</button>

<button (click)="removeCourse()">Remove Course</button>

<h3>Total Courses : {{courseCount$|async}}</h3>

<table border="1">

<tr>

<th>ID</th>
```

```

<th>Name</th>

<th>Code</th>

<th>Credits</th>

<th>Faculty</th>

</tr>

<tr *ngFor="let course of courses$|async">

<td>{{course.id}}</td>

<td>{{course.name}}</td>

<td>{{course.code}}</td>

<td>{{course.credits}}</td>

<td>{{course.faculty}}</td>

</tr>

</table>

```

Run

ng serve

Open:

http://localhost:4200

Output Screenshot:

The screenshots illustrate the application's state and actions:

- 1. Initial State (No Courses Loaded):** The application shows a 'Total Courses' count of 0. The Redux DevTools state is an empty array.
- 2. After Click Load Courses (Data Loaded into Store):** The application shows a 'Total Courses' count of 3. The Redux DevTools state contains three courses: Angular, React, and Spring Boot.
- 3. After Click Add Course (New Course Added to Store):** The application shows a 'Total Courses' count of 4. The Redux DevTools state contains four courses, including Java.
- 4. After Click Remove (Course Removed from Store):** The application shows a 'Total Courses' count of 3. The Redux DevTools state contains three courses, with Java removed.
- 5. Redux DevTools - Actions & State Changes:** A detailed view of the Redux DevTools interface showing the sequence of actions (Load Courses, Add Course, Remove Course) and the corresponding state changes.
- 6. Summary:** A summary of the application's state and actions, including a list of courses and a note about the application's use of NgRx Store.

NgRx Effects, Entity & DevTools

Install Required Packages

```
npm install @ngrx/effects @ngrx/entity
```

Generate Effect

```
ng generate effect store/course
```

course.actions.ts

```
import { createAction, props } from '@ngrx/store';
import { Course } from '../models/course.model';

export const loadCourses=createAction(
  '[Course] Load Courses'
);

export const loadCoursesSuccess=createAction(
  '[Course] Load Courses Success',
  props<{courses:Course[]}>()
);

export const loadCoursesFailure=createAction(
  '[Course] Load Courses Failure',
  props<{error:any}>()
);
```

course.effects.ts

```
import { Injectable } from '@angular/core';
import { Actions, createEffect, ofType } from '@ngrx/effects';
```

```
import { CourseService } from '../services/course.service';

import * as CourseActions from './course.actions';

import { catchError, map, mergeMap, of } from 'rxjs';

@Injectable()

export class CourseEffects{

  constructor(

    private actions$:Actions,

    private courseService:CourseService

  ){}

  loadCourses$=createEffect(()=>

    this.actions$.pipe(

      ofType(CourseActions.loadCourses),

      mergeMap(()=>

        this.courseService.getCourses().pipe(

          map(courses=>

            CourseActions.loadCoursesSuccess({

              courses

            })

          ),

          catchError(error=>

            of(

              CourseActions.loadCoursesFailure({

                error

              })

            )

          )

        )

      )

    )

  )
```

```
)  
)  
);  
}
```

course.reducer.ts

```
import { createReducer,on } from '@ngrx/store';  
import { EntityState,EntityAdapter,createEntityAdapter } from '@ngrx/entity';  
import * as CourseActions from './course.actions';  
import { Course } from '../models/course.model';  
export interface CourseState extends EntityState<Course>{  
  loading:boolean;  
}  
export const adapter:EntityAdapter<Course>=  
  createEntityAdapter<Course>();  
export const initialState:CourseState=  
  adapter.getInitialState({  
    loading:false  
  });  
export const courseReducer=createReducer(  
  initialState,  
  on(CourseActions.loadCourses,state=>({  
    ...state,  
    loading:true  
  })),  
  on(CourseActions.loadCoursesSuccess,(state,{courses})=>  
    adapter.setAll(courses,{  
      ...state,
```

```
loading:false
})
)
);
```

course.selectors.ts

```
import { createFeatureSelector } from '@ngrx/store';
import { adapter, CourseState } from './course.reducer';
export const selectCourseState=
createFeatureSelector<CourseState>('courses');
export const{
selectAll,
selectEntities,
selectIds,
selectTotal
}=adapter.getSelectors(selectCourseState);
```

app.config.ts

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideStore } from '@ngrx/store';
import { provideEffects } from '@ngrx/effects';
import { provideStoreDevtools } from '@ngrx/store-devtools';
import { routes } from './app.routes';
import { courseReducer } from './store/course.reducer';
import { CourseEffects } from './store/course.effects';
export const appConfig:ApplicationConfig={
providers:[
```

```
provideRouter(routes),
provideStore({
  courses:courseReducer
}),
provideEffects([
  CourseEffects
]),
provideStoreDevtools({
  maxAge:25
})
]
};
```

course-list.component.ts

```
import { Component } from '@angular/core';
import { Store } from '@ngrx/store';
import { Observable } from 'rxjs';
import { Course } from '../models/course.model';
import * as CourseActions from '../store/course.actions';
import * as CourseSelectors from '../store/course.selectors';
@Component({
  selector:'app-course-list',
  standalone:true,
  templateUrl:'./course-list.component.html',
  styleUrls:['./course-list.component.css']
})
export class CourseListComponent{
  courses$:Observable<Course[]>;
```

```
total$:Observable<number>;

constructor(private store:Store){

this.courses$=

this.store.select(

CourseSelectors.selectAll

);

this.total$=

this.store.select(

CourseSelectors.selectTotal

);

}

loadCourses(){

this.store.dispatch(

CourseActions.loadCourses()

);

}

}
```

course-list.component.html

```
<h2>NgRx Effects Demo</h2>

<button (click)="loadCourses()">
Load Courses
</button>

<h3>Total Courses : {{total$|async}}</h3>

<table border="1">

<tr>

<th>ID</th>
```

```

<th>Name</th>

<th>Code</th>

<th>Credits</th>

<th>Faculty</th>

</tr>

<tr *ngFor="let course of courses$|async">

<td>{{course.id}}</td>

<td>{{course.name}}</td>

<td>{{course.code}}</td>

<td>{{course.credits}}</td>

<td>{{course.faculty}}</td>

</tr>

</table>

```

Run

ng serve

Open:

http://localhost:4200

Output Screenshot:

The screenshot displays the application's output across three main components:

- Student Course Portal (Web Application):** Shows a table of courses with columns ID, Course Name, Code, Credits, Faculty, and Actions. The table lists 5 courses: Angular, React, Spring Boot, Java, and MySQL. A green message at the bottom states "Courses loaded successfully from store."
- Redux DevTools:** Shows the state and actions. The state is an array of 5 course objects. The actions include "Load Courses" and "Load Courses Success".
- Selectors Output:** Shows the output of the selectors. The "selectAllCourses" selector returns an array of 5 course objects. The "selectCourseCount" selector returns the count 5.